

Phương pháp tham lam

Bài giảng "*Thiết kế và đánh giá thuật toán*"

Ngày 6 tháng 4 năm 2026

Nội dung

- 1 Giới thiệu
- 2 Một số bài toán điển hình
- 3 Bài toán cây bao trùm tối thiểu
- 4 Mã Huffman

Giới thiệu

- Cho trước một tập A gồm n đối tượng, ta cần phải chọn một tập con S từ tập A . Với mỗi tập con S được chọn ra thỏa mãn các yêu cầu của bài toán, ta gọi là một nghiệm chấp nhận được. Nghiệm tối ưu là nghiệm chấp nhận được mà tại đó hàm mục tiêu đạt giá trị nhỏ nhất (lớn nhất).

Giới thiệu

- Cho trước một tập A gồm n đối tượng, ta cần phải chọn một tập con S từ tập A . Với mỗi tập con S được chọn ra thỏa mãn các yêu cầu của bài toán, ta gọi là một nghiệm chấp nhận được. Nghiệm tối ưu là nghiệm chấp nhận được mà tại đó hàm mục tiêu đạt giá trị nhỏ nhất (lớn nhất).
- *Ý tưởng tham lam*: Xây dựng lời giải của bài toán với việc chấp nhận những lựa chọn có vẻ tốt nhất của từng giai đoạn (chấp nhận lựa chọn tối ưu địa phương/ cục bộ).

Giới thiệu

- Cho trước một tập A gồm n đối tượng, ta cần phải chọn một tập con S từ tập A . Với mỗi tập con S được chọn ra thỏa mãn các yêu cầu của bài toán, ta gọi là một nghiệm chấp nhận được. Nghiệm tối ưu là nghiệm chấp nhận được mà tại đó hàm mục tiêu đạt giá trị nhỏ nhất (lớn nhất).
- *Ý tưởng tham lam*: Xây dựng lời giải của bài toán với việc chấp nhận những lựa chọn có vẻ tốt nhất của từng giai đoạn (chấp nhận lựa chọn tối ưu địa phương/ cục bộ).
- *Những bài toán có thể giải bằng phương pháp tham lam*.

Giới thiệu

- Cho trước một tập A gồm n đối tượng, ta cần phải chọn một tập con S từ tập A . Với mỗi tập con S được chọn ra thỏa mãn các yêu cầu của bài toán, ta gọi là một nghiệm chấp nhận được. Nghiệm tối ưu là nghiệm chấp nhận được mà tại đó hàm mục tiêu đạt giá trị nhỏ nhất (lớn nhất).
- *Ý tưởng tham lam*: Xây dựng lời giải của bài toán với việc chấp nhận những lựa chọn có vẻ tốt nhất của từng giai đoạn (chấp nhận lựa chọn tối ưu địa phương/ cục bộ).
- *Những bài toán có thể giải bằng phương pháp tham lam*.
 - Bài toán có lời giải tối ưu;

Giới thiệu

- Cho trước một tập A gồm n đối tượng, ta cần phải chọn một tập con S từ tập A . Với mỗi tập con S được chọn ra thỏa mãn các yêu cầu của bài toán, ta gọi là một nghiệm chấp nhận được. Nghiệm tối ưu là nghiệm chấp nhận được mà tại đó hàm mục tiêu đạt giá trị nhỏ nhất (lớn nhất).
- *Ý tưởng tham lam*: Xây dựng lời giải của bài toán với việc chấp nhận những lựa chọn có vẻ tốt nhất của từng giai đoạn (chấp nhận lựa chọn tối ưu địa phương/ cục bộ).
- *Những bài toán có thể giải bằng phương pháp tham lam*.
 - Bài toán có lời giải tối ưu;
 - Bài toán trải qua nhiều bước, và tại mỗi bước có thể tìm được lời giải tối ưu cho từng bài toán nhỏ;

Giới thiệu

- Cho trước một tập A gồm n đối tượng, ta cần phải chọn một tập con S từ tập A . Với mỗi tập con S được chọn ra thỏa mãn các yêu cầu của bài toán, ta gọi là một nghiệm chấp nhận được. Nghiệm tối ưu là nghiệm chấp nhận được mà tại đó hàm mục tiêu đạt giá trị nhỏ nhất (lớn nhất).
- *Ý tưởng tham lam*: Xây dựng lời giải của bài toán với việc chấp nhận những lựa chọn có vẻ tốt nhất của từng giai đoạn (chấp nhận lựa chọn tối ưu địa phương/ cục bộ).
- *Những bài toán có thể giải bằng phương pháp tham lam*.
 - Bài toán có lời giải tối ưu;
 - Bài toán trải qua nhiều bước, và tại mỗi bước có thể tìm được lời giải tối ưu cho từng bài toán nhỏ;
 - Lời giải của bài toán sẽ được bổ sung từng bước thông qua lời giải của bài toán con.

Giới thiệu

- Cho trước một tập A gồm n đối tượng, ta cần phải chọn một tập con S từ tập A . Với mỗi tập con S được chọn ra thỏa mãn các yêu cầu của bài toán, ta gọi là một nghiệm chấp nhận được. Nghiệm tối ưu là nghiệm chấp nhận được mà tại đó hàm mục tiêu đạt giá trị nhỏ nhất (lớn nhất).
- *Ý tưởng tham lam*: Xây dựng lời giải của bài toán với việc chấp nhận những lựa chọn có vẻ tốt nhất của từng giai đoạn (chấp nhận lựa chọn tối ưu địa phương/ cục bộ).
- *Những bài toán có thể giải bằng phương pháp tham lam*.
 - Bài toán có lời giải tối ưu;
 - Bài toán trải qua nhiều bước, và tại mỗi bước có thể tìm được lời giải tối ưu cho từng bài toán nhỏ;
 - Lời giải của bài toán sẽ được bổ sung từng bước thông qua lời giải của bài toán con.

Giới thiệu

- *Tính chất của thuật toán tham lam*

Giới thiệu

- *Tính chất của thuật toán tham lam*
 - Tính chất của sự lựa chọn tham lam: Một giải pháp tối ưu toàn cục có thể được giải bằng cách thực hiện những lựa chọn tối ưu cục bộ.

Giới thiệu

- *Tính chất của thuật toán tham lam*

- Tính chất của sự lựa chọn tham lam: Một giải pháp tối ưu toàn cục có thể được giải bằng cách thực hiện những lựa chọn tối ưu cục bộ.
- Tính chất cấu trúc con tối ưu: Một giải pháp tối ưu của bài toán chứa trong nó những giải pháp tối ưu cho các bài toán con của nó;

- *Mô hình*

Chọn S từ tập A

- $S = \emptyset$;
- Trong khi $A \neq \emptyset$:
 - Chọn phần tử x tốt nhất của A ;
 - Cập nhật các đối tượng để chọn $A = A - \{x\}$;
 - Nếu $S \cup \{x\}$ thỏa mãn điều kiện của bài toán thì cập nhật lời giải $S = S \cup \{x\}$

Giới thiệu

Lược đồ

```
input A[1...n];  
output S;  
greedy (A, n)  
    S =  $\emptyset$ ;  
    while ( $A \neq \emptyset$ ){  
        x = Chon(A);  
        A = A - {x};  
        if ( $S \cup \{x\}$  chấp nhận được)  
            S = S  $\cup$  {x};  
    }
```

Nội dung

- 1 Giới thiệu
- 2 Một số bài toán điển hình
- 3 Bài toán cây bao trùm tối thiểu
- 4 Mã Huffman

Một số bài toán điển hình

Bài toán người đưa hàng

Bài toán: Một người bán hàng muốn tham quan n thành phố $T_1, T_2, \dots, T_n$. Xuất phát từ một thành phố nào đó, người bán hàng muốn đi qua tất cả các thành phố còn lại. Mỗi thành phố đi qua đúng 1 lần rồi quay trở lại thành phố xuất phát. Gọi C_{ij} là chi phí đi từ thành phố T_i đến thành phố T_j .

Yêu cầu: Tìm một hành trình thỏa mãn yêu cầu bài toán sao cho tổng chi phí là nhỏ nhất.

Một số bài toán điển hình

Bài toán người đưa hàng

Ý tưởng:

- Đây là bài toán tìm chu trình có trọng số nhỏ nhất trong một đơn đồ thị có hướng có trọng số;
- Thuật toán tham lam cho bài toán là chọn thành phố có chi phí nhỏ nhất tính từ thành phố hiện thời đến các thành phố chưa qua.

Một số bài toán điển hình

Bài toán người đưa hàng

Các bước tiến hành thuật toán

- 1 Chọn một đỉnh bắt đầu v ;
- 2 Từ đỉnh hiện hành chọn cạnh nối có chiều dài nhỏ nhất đến các đỉnh chưa đi qua. Đánh dấu đã đi qua đỉnh vừa chọn;
- 3 Nếu còn đỉnh chưa đi qua thì quay lại bước 2;
- 4 Quay lại đỉnh v .

Một số bài toán điển hình

Bài toán người đưa hàng

input: $c = c_{ij}$

output: X // hành trình tối ưu;
cost // chi phí tối ưu

Mô tả:

daxet[n]; // đánh dấu các đỉnh được sử dụng;
min; // Chọn min các cạnh trong mỗi bước

Một số bài toán điển hình

Bài toán người đưa hàng

```
greedy_TSP(c, n)
  while (i < n){
    min =  $\infty$ ;
    for (k = 1; k <=n; k++){
      if (!daxet[k]){
        if (min > c[v][k]){
          min = c[v][k];
          w = k;
        }
      }
    }
    v = w;
    i++; x[i] = v;
    daxet[v] = 1;
    cost += min;
```

Một số bài toán điển hình

Bài toán cái túi

Bài toán: Có n loại đồ vật, mỗi loại có số lượng không hạn chế. Đồ vật loại i có trọng lượng w_i và giá trị sử dụng v_i , $i \in \{1, \dots, n\}$.

Yêu cầu: Chọn các đồ vật để đặt vào túi có giới hạn trọng lượng m , sao cho tổng giá trị lựa chọn là lớn nhất.

Một số bài toán điển hình

Bài toán cái túi

Áp dụng thuật toán tham lam

- Tính “đơn giá” ($\text{đơn giá} = \frac{v_i}{w_i}$) cho từng loại đồ vật;
- Xếp theo đơn giá giảm dần;
- Với mỗi loại đồ vật sẽ lấy số lượng tối đa mà trọng lượng của túi còn cho phép;
- Xác định lại trọng lượng túi, quay lại bước 3 cho đến khi không bỏ thêm vào được nữa;

Một số bài toán điển hình

Bài toán cái túi

```
greedy_backpack()  
    sort(); //Sắp xếp các đồ vật giảm dần theo  $\frac{v_i}{w_i}$   
    sum = 0;  
    for (i=1; i<=n; i++){  
        if (m >= w[i]){  
            x[i] = (int)m / w[i];  
            m -= x[i]*w[i] ;  
            sum += x[i]*v[i] ;  
        }  
    }
```

Một số bài toán điển hình

Bài toán đổi tiền

Bài toán: Có một lượng tiền M đơn vị (đồng), và k đồng tiền mệnh giá giảm dần lần lượt $m_1, m_2, \dots, m_k$. Với giả thiết số lượng các đồng tiền là không hạn chế.

Yêu cầu: Hãy đổi M lượng tiền thành các đồng tiền sao cho số lượng các tờ tiền là ít nhất.

Một số bài toán điển hình

Bài toán đổi tiền

- Gọi $X = (x_1, x_2, \dots, x_k)$ là một phương án trả tiền.
- Mục tiêu $x_1 + x_2 + \dots + x_k$ nhỏ nhất, với ràng buộc
$$x_1 * m_1 + x_2 * m_2 + \dots + x_k * m_k = M$$

Ý tưởng

- Để $x_1 + x_2 + \dots + x_k$ nhỏ nhất thì các đồng tiền có mệnh giá lớn nhất phải được chọn nhiều nhất.
- Không mất tính tổng quát, giả sử các đồng tiền đã được sắp xếp giảm dần $m_1 > m_2 > \dots > m_k$.
- Trước hết chọn tối đa các đồng tiền có mệnh giá m_1 , x_1 là số nguyên lớn nhất sao cho $x_1 * m_1 \leq M$.
- Số tiền còn lại là $M - x_1 * m_1$.
- Chuyển sang chọn các đồng tiền có mệnh giá nhỏ hơn,....

Một số bài toán điển hình

Bài toán đổi tiền

```
greedy_ATM(m, M)
  for (i = 0; i < k; i++){
    if (M >= m[i]){
      x[i] = M/m[i];
      M = M - m[i]*x[i];
    }
  }
  if (M != 0)
    printf("Khong co cach doi tien");
  else
    inNghiem(x);
```

Nội dung

- 1 Giới thiệu
- 2 Một số bài toán điển hình
- 3 Bài toán cây bao trùm tối thiểu**
- 4 Mã Huffman

Giới thiệu bài toán

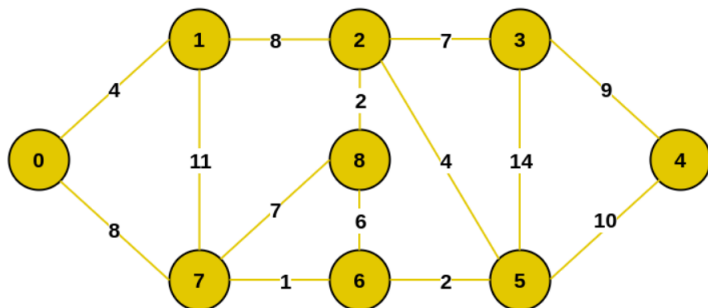
Bài toán cây khung (cây bao trùm) nhỏ nhất của đồ thị là một trong số những bài toán tối ưu trên đồ thị có nhiều ứng dụng khác nhau trong đời sống.

- Bài toán xây dựng đường giao thông: giả sử ta muốn xây dựng một hệ thống đường nối n thành phố sao cho giữa các thành phố bất kì luôn có đường đi. Bài toán đặt ra là tìm cây khung nhỏ nhất trên đồ thị, mỗi thành phố ứng với một đỉnh sao cho tổng chi phí xây dựng đường đi là nhỏ nhất.
- Bài toán nối mạng máy tính: Cần nối mạng một hệ thống gồm n máy tính đánh số từ 1 đến n . Biết chi phí nối máy i với máy j là $c[i, j]$, $i, j = 1, 2, \dots, n$ (thông thường chi phí này phụ thuộc vào độ dài cáp nối cần sử dụng). Hãy tìm cách nối mạng sao cho tổng chi phí nối mạng là nhỏ nhất.

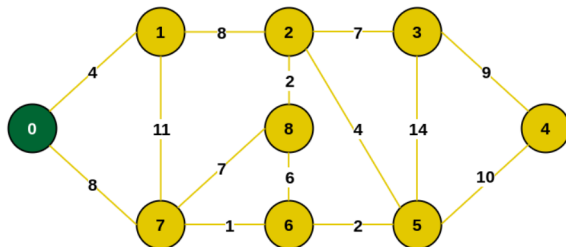
Thuật toán Prim

Ý tưởng thuật toán:

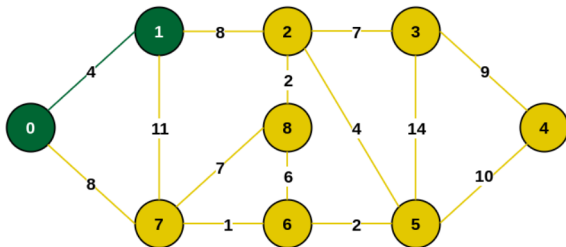
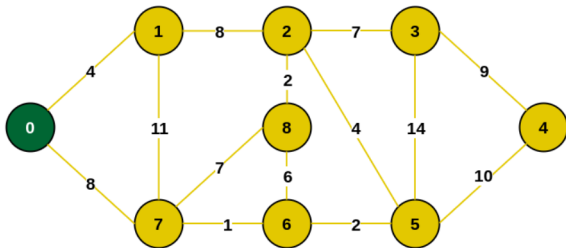
- Tại mỗi bước: chọn một đỉnh chưa được xét, sao cho đỉnh đó kề và gần nhất với các đỉnh đã được xét
- Ví dụ minh họa:



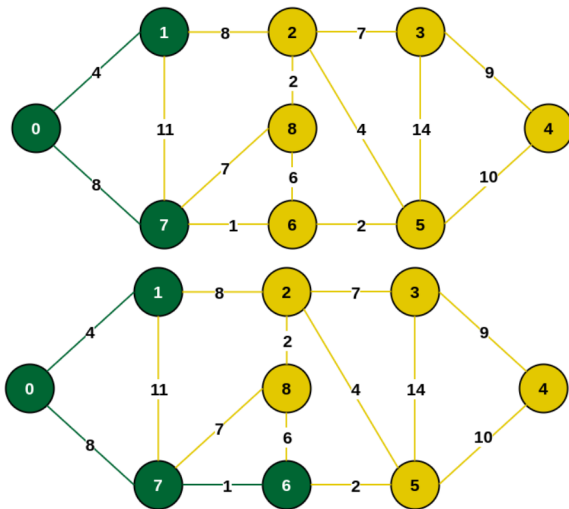
Ví dụ



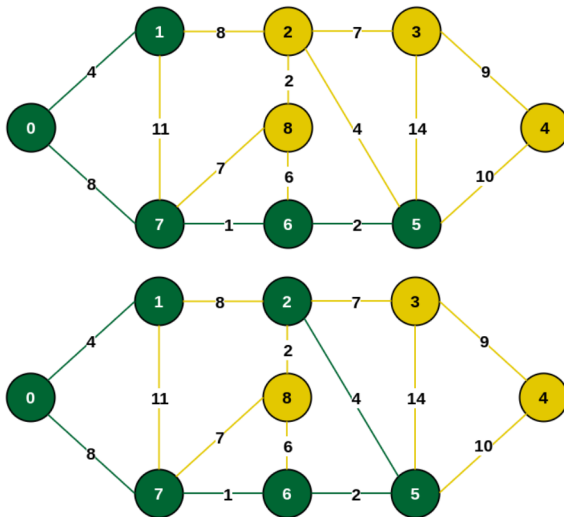
Ví dụ



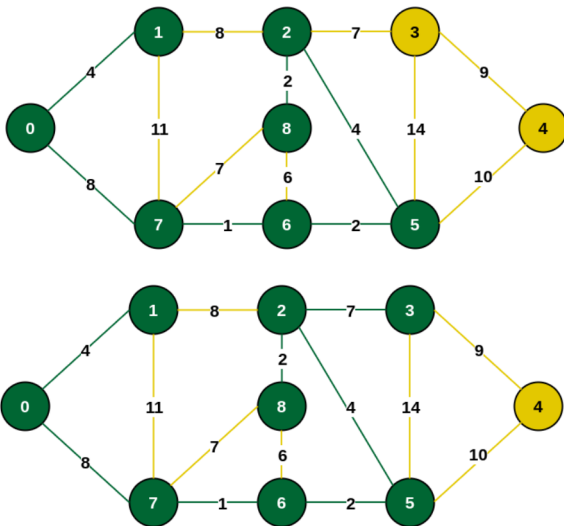
Ví dụ



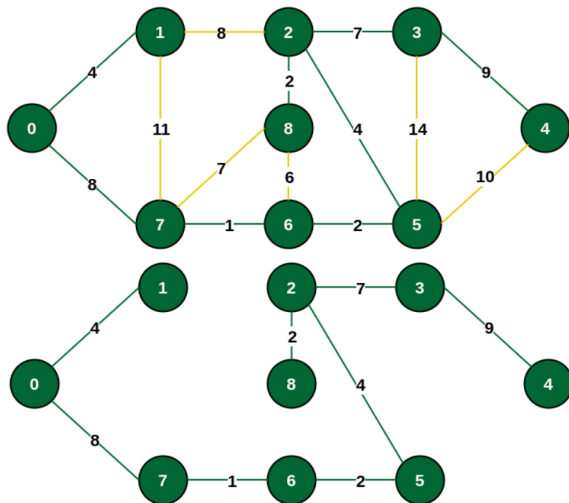
Ví dụ



Ví dụ



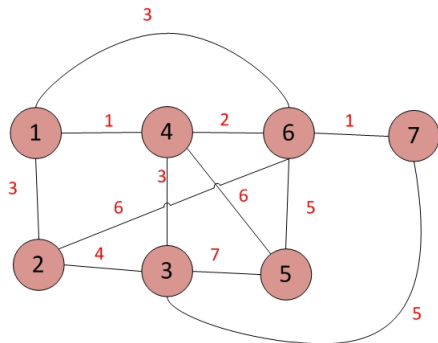
Ví dụ



Thuật toán Kruskal

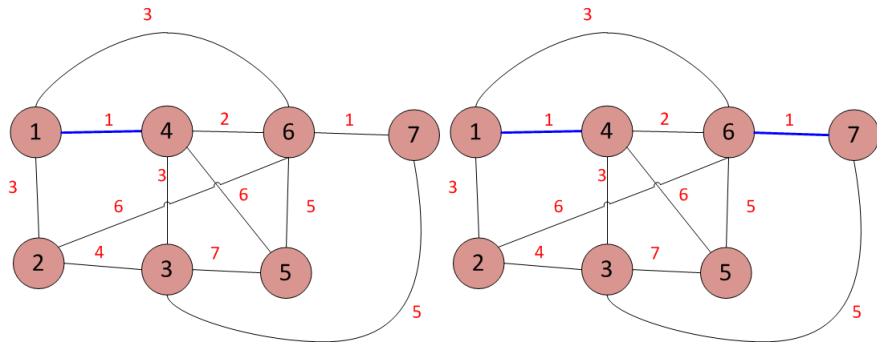
- Bước 1: Sắp xếp các cạnh của đồ thị theo thứ tự trọng số tăng dần.
- Bước 2: Khởi tạo $T := \emptyset$
- Bước 3: Lần lượt lấy từng cạnh thuộc danh sách đã sắp xếp. Nếu $T \cup \{e\}$ không chứa chu trình thì gán $T := T \cup \{e\}$.
- Bước 4: Nếu T đủ $n - 1$ phần tử thì dừng, ngược lại làm tiếp bước 3.

Ví dụ

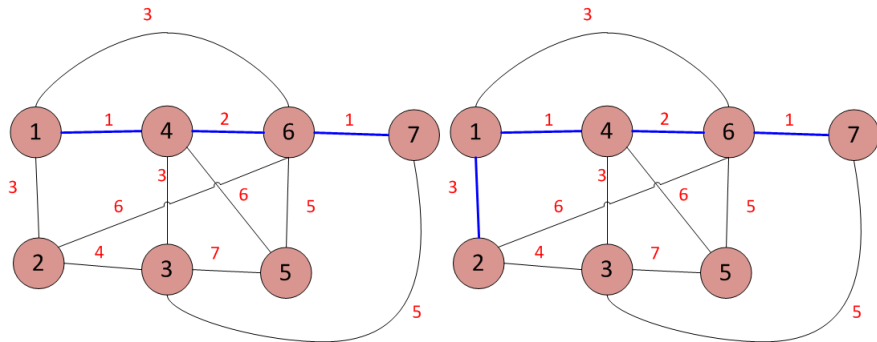


Điểm đầu	Điểm cuối	Trọng số
1	4	1
6	7	1
4	6	2
1	2	3
1	6	3
3	4	3
2	3	4
3	7	5
5	6	5
2	6	6
4	5	6
3	5	7

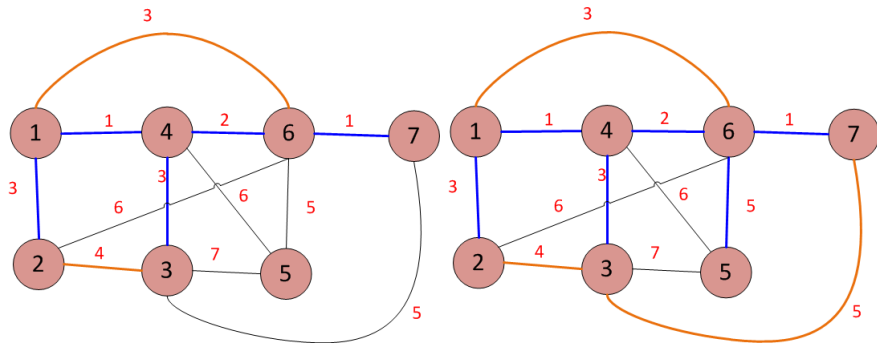
Ví dụ



Ví dụ



Ví dụ



Nội dung

- 1 Giới thiệu
- 2 Một số bài toán điển hình
- 3 Bài toán cây bao trùm tối thiểu
- 4 Mã Huffman**

Mã Huffman

Mã Huffman

Giả sử trong văn bản có bảng chữ cái C , với mỗi chữ cái $c \in C$ ta biết tần suất xuất hiện của nó trong văn bản là $f(c)$. Cần tìm cách mã hoá văn bản sử dụng ít bộ nhớ nhất.

Hai cách mã hoá phổ biến

- Mã hoá với độ dài cố định: Bảng mã ASCII mở rộng dùng 8 bit để biểu diễn 256 ký tự khác nhau
- Mã phi tiền tố (prefix free code): mỗi ký tự c được mã hoá bởi một xâu nhị phân $code(c)$ sao cho mã của ký tự bất kì không là đoạn đầu của bất cứ mã của ký tự nào khác trong các ký tự còn lại. Tuy nhiên, cách mã hoá này đòi hỏi việc mã hoá và giải mã phức tạp

Mã Huffman (1)

Mã Huffman

Mỗi mã phi tiền tố có thể biểu diễn bởi một cây nhị phân T mà mỗi lá của nó tương ứng với một chữ cái và cạnh của nó được gán cho một trong hai số 1 và 0. Mã của một chữ cái c là một dãy nhị phân gồm các số gán cho các cạnh trên đường đi từ gốc đến c

Bài toán

Tìm cách mã hoá tối ưu, tức là cây nhị phân T làm tối thiểu hoá tổng độ dài có trọng số

$$B(T) = \sum_{c \in C} f(c) \times \text{depth}(c)$$

trong đó, $\text{depth}(c)$ là độ dài đường đi từ gốc đến lá tương ứng với ký tự c , $f(c)$ là tần số xuất hiện của ký tự c

Mã Huffman (2)

Thuật toán

Ý tưởng

Chữ cái có tần suất xuất hiện ít hơn cần được gán cho lá có khoảng cách đến gốc là lớn hơn. Ngược lại, chữ cái có tần suất xuất hiện nhiều cần được gán cho nút gần gốc

Mã Huffman (3)

Thuật toán

Với ý tưởng trên, thuật toán Huffman coding gồm 3 bước:

- ➊ Đếm tần suất xuất hiện của các phần tử trong chuỗi đầu vào.
- ➋ Xây dựng cây Huffman (cây nhị phân mã hóa).
- ➌ Từ cây Huffman, ta có được các giá trị mã hóa. Lúc này, ta có thể xây dựng chuỗi mã hóa từ các giá trị này.

Quá trình xây dựng cây Huffman gồm các bước sau:

- ➊ Tạo danh sách chứa các nút lá bao gồm phần tử đầu vào và trọng số nút là tần suất xuất hiện của nó.
- ➋ Từ danh sách này, lấy ra 2 phần tử có tần suất xuất hiện ít nhất. Sau đó gán 2 nút vừa lấy ra vào một nút gốc mới với trọng số là tổng của 2 trọng số ở nút vừa lấy ra để tạo thành một cây.
- ➌ Đẩy cây mới vào lại danh sách.
- ➍ Lặp lại bước 2 và 3 cho đến khi danh sách chỉ còn 1 nút gốc duy nhất của cây.
- ➎ Nút còn lại chính là nút gốc của cây Huffman.

Mã Huffman (4)

Ví dụ

Cho xâu “DHKHTN” với số lần xuất hiện của các kí tự giảm dần như sau:

Ký tự	‘H’	‘D’	‘K’	‘T’	‘N’
Tần suất	2	1	1	1	1

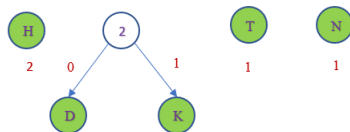
- Mỗi ký tự sẽ là một nút lá, kèm theo trọng số là tần suất xuất hiện của nó
- 2 nút có trọng số thấp nhất sẽ được chọn để dựng cây tại mỗi bước

Mã Huffman (5)

Ví dụ

Cho xâu “DHKHTN” với số lần xuất hiện của các kí tự giảm dần như sau:

Ký tự	‘H’	‘DK’	‘T’	‘N’
Tần suất	2	2	1	1

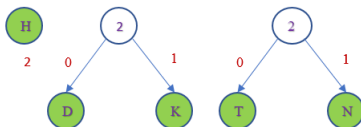


Mã Huffman (6)

Ví dụ

Cho xâu “DHKHTN” với số lần xuất hiện của các kí tự giảm dần như sau:

Kí tự	‘H’	‘DK’	‘TN’
Tần suất	2	2	2

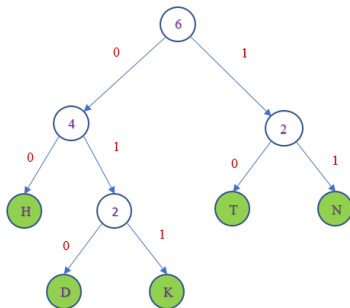


Mã Huffman (7)

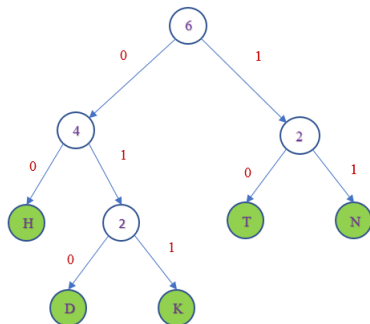
Ví dụ

Kết quả mã Huffman cho các ký tự trong văn bản “DHKHTN”

Ký tự	‘H’	‘D’	‘K’	‘T’	‘N’
Mã Huffman	00	010	011	10	11

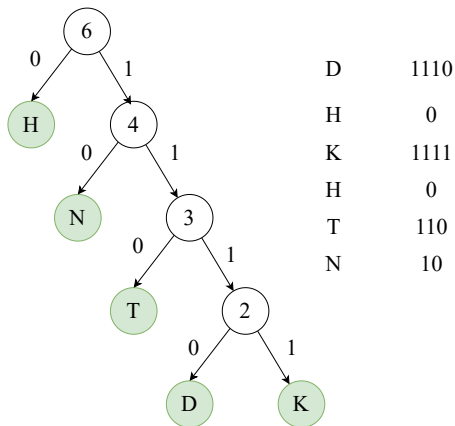


Mã Huffman (8)



Trong mỗi bước của thuật toán xây dựng cây Huffman, ta luôn phải chọn ra hai gốc có trọng số nhỏ nhất. Để làm việc này ta sắp xếp các gốc vào một hàng đợi ưu tiên theo tiêu chuẩn trọng số nhỏ nhất. Một trong các cấu trúc dữ liệu thuận lợi cho tiêu chuẩn này là cấu trúc đống (với phần tử có trọng số nhỏ nhất nằm trên đỉnh của đống).

Mã Huffman (9)



Hình 1: Một ví dụ minh họa khác.